

Zero to Hero Study Handbook: Medical Research GraphRAG

Module 1: Foundations & Architecture

1.1 What this project does

This repository is an end-to-end biomedical RAG system built around real MedMentions records, with multiple retrieval/generation variants:

- Baseline GraphRAG with ChromaDB (`notebooks/NB02_Chroma_GraphRAG.py`, `src/chroma_retriever.py`, `src/graph_builder.py`)
- Pinecone GraphRAG benchmarking (`notebooks/NB03_Pinecone_GraphRAG.py`, `src/pinecone_retriever.py`)
- Agentic GraphRAG with LangGraph routing (`notebooks/NB04_Agentic_GraphRAG.py`, `src/agentic_rag.py`)
- Unified evaluation across retrieval/generation/RAG/judge metrics (`notebooks/NB05_Evaluation.py`, `src/evaluator.py`)
- Hybrid dense+sparse retrieval (`notebooks/NB06_Hybrid_RAG.py`, `src/hybrid_retriever.py`)
- Corrective RAG (CRAG) with correction loops (`notebooks/NB07_CRAG.py`, `src/crag_pipeline.py`)
- Multimodal OCR and vision paths (`notebooks/NB08/NB09/NB10`, `src/multimodal_rag.py`, `src/multimodal_vision_rag.py`)
- Optional selective fine-tuning track (`notebooks/NB11_Selective_Finetuning_Unsloth_PEFT_TRL.py`, `src/finetune_data.py`, `src/finetune_unsloth.py`)

The strict orchestrator script is `scripts/run_full_real_pipeline_strict.sh`, which runs preflight, NB01-NB11, and `pytest -q` with resumable state in `outputs/run_state/full_real_pipeline.state`.

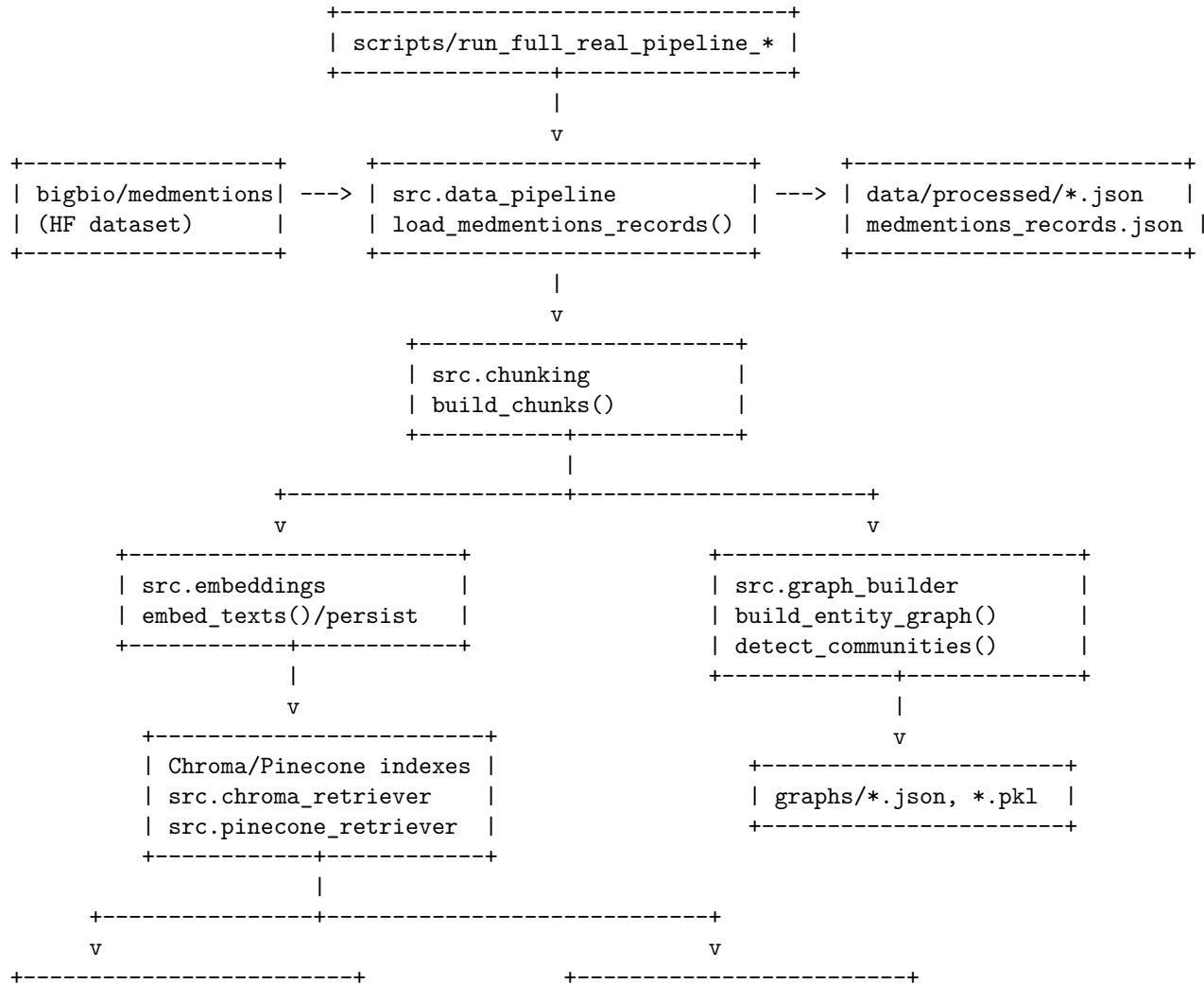
1.2 Core paradigms and patterns used here

Definitions first, then where they appear:

- Data pipeline pattern: staged transform from raw dataset to normalized records, chunks, embeddings, graph artifacts.
 - Implemented in `src/data_pipeline.py`, `src/chunking.py`, `src/embeddings.py`, `src/graph_builder.py`.
- Dataclass-based domain modeling: typed data contracts for pipeline entities.
 - `EntityMention`, `MedRecord`, `EvalQuery`, `ChunkRecord`, `RelationEdge`, `GenerationExample`, etc.
- Functional utility style: many pure-ish functions for deterministic transformations.
 - Example: `build_chunks`, `compute_retrieval_metrics`, `build_multimodal_eval_queries`.
- State-machine orchestration: explicit node graphs and conditional branching.

- `build_agentic_workflow` in `src/agentic_rag.py`, `build_crag_workflow` in `src/crag_pipeline.py`.
- Hybrid retrieval fusion: combining dense and sparse channels.
 - `hybrid_search`, `weighted_score_fusion`, and optional `reciprocal_rank_fusion`.
- Configuration-as-code with env-backed settings: centralized runtime control.
 - `ProjectSettings` in `src/config.py` (Pydantic Settings over `.env`).
- Placeholder-safe reporting pattern in additive notebooks (NB06+):
 - If `RUN_FULL_EVAL/RUN_FULL_TRAIN` is false, schema-complete JSON payloads are still written with placeholder values.

1.3 Architecture (high-level)



File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/data_pipeline.py	MedMentions normalization + eval query generation	EntityMention, MedRecord, EvalQuery, load_medmentions_records, build_extractive_eval_queries	max_records, random_seed
src/chunking.py	Text chunking + entity metadata propagation	ChunkRecord, recursive_split, build_chunks, build_chunk_lookup	chunk_size, chunk_overlap
src/embeddings.py	Ollama embedding operations + persistence	EmbeddingBundle, embed_texts, embed_query, persist_embedding_bundle	embedding_model, OLLAMA_EMBED_TIMEOUT_SECONDS
src/chroma_retrieval.py	Local vector DB indexing and retrieval	index_chunks_to_embeddings, vector_search, entity_search, reciprocal_rank_fusion	collection names, top_k
src/graph_builder.py	Graph construction, relation extraction, community ops	RelationEdge, build_entity_graph, extract_relationship_edges, detect_communities	local_graph_hops
src/pinecone_retrieval.py	Managed vector indexing/query + cost proxy	create_index, index_chunks_to_pinecone, query_pinecone, pinecone_cost_proxy	PINECONE_API_KEY, pinecone_region/prefix
src/agent_rag.py	LangGraph agentic workflow	AgentState, AgentResources, build_agentic_workflow, run_agentic_query	retrieval_grade_threshold, hallucination_threshold
src/llm_judge.py	JSON-based judge wrappers	judge_json, grade_retrieval_quality, grade_groundedness	OLLAMA_JUDGE_RETRIES, OLLAMA_JUDGE_TIMEOUT_SECONDS
src/guardian_judge.py	Backward-compatible judge shim	guardian_json	guardian_judge_model alias behavior
src/evaluator.py	Retrieval/generation metric engine	RetrievalExample, GenerationExample, EvaluationBundle, build_evaluation_bundle	judge_model, BERTScore, toggle
src/hybrid_retrieval.py	Sparse biomedical retriever + hybrid fusion	BiomedicalSparseIndex, hybrid_search, weighted_score_fusion	hybrid_dense_weight, hybrid_sparse_weight

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/crag_pipeline.py	Corrective RAG state machine	CragState, CragResources, build_crag_workflow, run_crag_query	crag_acceptance_threshold, crag_max_corrections
src/multimodal_assets.py	Fetch PMC multimodal assets and manifest from OVID datasets	PMCMultimodalAssets, fetch_pmc_multimodal_assets	max_images, max_tables
src/multimodal_rag.py	OCR/table multimodal ingestion + retrieval	MultimodalDocument, MultimodalChunk, ocr_cli_*, build_multimodal_documents, multimodal_vector_search	multimodal_ocr_model, ocr_cli_*
src/multimodal_vision_rag.py	Vision rag by extraction multimodal path	extract_vision_evidence, build_vision_multimodal_document, MultimodalDocument, MULTIMODAL_VISION_RETRIES	multimodal_vision_model, MULTIMODAL_VISION_RETRIES
src/finetune_data.py	Build SFT datasets from real eval queries	SFTExample, build_biomedical_sft_examples, persist_sft_jsonl	finetune_max_*
src/finetune_unsloth.py	Optional training/adapters/export helpers	LoRAHyperParams, SFTTrainConfig, create_unsloth_loRa_model, create_sft_trainer	finetune_*, optional package availability
scripts/execute_nb.py	Execute baseline notebook subset (NB01-NB05)	run_nb shell function	RUN_FULL_EVAL, RUN_FULL_TRAIN, OLLAMA_HOST
scripts/execute_additional_nb.py	Execute additional notebook subset (NB06-NB11)	run_nb shell function	same vars as above
scripts/run_full_pipeline.py	Full pipeline, batch orchestrator with retries	main, run_once_with_retry, run_notebook	MAX_RETRIES, NOTEBOOK_TIMEOUT, CLEANUP_PINECONE_INDEX
scripts/fetch_pmc_multimodal_assets.py	Full wrapped fetch multimodal asset fetch	main, parse_args	--max-images, --max-tables
notebooks/NB01_*.py ... NB11_*.py	Tutorial-grade runnable pipeline stages	notebook-level functions and payload builders	RUN_FULL_EVAL, RUN_FULL_TRAIN gates
tests/	Behavioral and contract tests	module-specific tests + notebook contract tests	expected section markers, payload contract checks

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
<code>outputs/metrics/</code>	Persisted metric artifacts used in docs/results	JSON payloads per notebook	<code>mode</code> (<code>executed</code> or <code>placeholder</code>)
<code>outputs/tables/</code>	Persisted summary tables	CSV outputs from notebooks	metric/table schema
<code>graphs/</code>	Persisted graph/community artifacts	<code>community_partitioning</code> <code>community_summaries</code>	graph json use for NB03-NB05

Module 3: Core Execution Flows

3.1 Flow A: Full strict batch pipeline (main entrypoint)

Entry file: `scripts/run_full_real_pipeline_strict.sh`

Step-by-step

1. `main()` ensures `.venv`, activates it, runs `uv sync --extra dev --extra finetune`.
2. Exports runtime vars: `OLLAMA_HOST`, `RUN_FULL_EVAL=true`, `RUN_FULL_TRAIN=true`, `CLEANUP_PINECONE_INDEX` default false.
3. Requires `PINECONE_API_KEY` for Pinecone notebook sections.
4. Runs preflight via `run_once_with_retry`:
 - `ensure_ollama_service`
 - `ensure_model` for `qwen3-embedding:4b`, `granite4.1:8b`, `glm-ocr`, `qwen3.5:4b`
 - `python scripts/fetch_pmc_multimodal_assets.py --max-images 5 --max-tables 3`
5. Runs notebooks in order with `run_notebook` and state tracking keys like `nb_NB05_Evaluation`.
6. Runs `pytest -q` via `run_once_with_retry` key `pytest_q`.
7. Optionally runs Pinecone cleanup Python block if `CLEANUP_PINECONE_INDEX=true`.

Key input/output shapes

- State file: `outputs/run_state/full_real_pipeline.state`
 - Shape: line-oriented `key=value` pairs (for example `nb_NB07_CRAG=done`).
- Run logs: `outputs/logs/full_real_pipeline_<timestamp>.log`
- Notebook outputs: `notebooks/<NB>.executed.ipynb`

3.2 Flow B: Data foundation -> chunks -> embeddings -> graph artifacts

Primary modules: `src/data_pipeline.py`, `src/chunking.py`, `src/embeddings.py`, `src/graph_builder.py`

Step-by-step

1. Load and normalize dataset records:

```
records = load_medmentions_records(max_records=settings.max_records)
```

2. Persist records once for reuse:

```
persist_records(records, settings.processed_dir / "medmentions_records.json")
```

3. Build chunk-level retrievable units:

```
chunks = build_chunks(records, chunk_size=settings.chunk_size, chunk_overlap=settings.chunk_overlap)
```

4. Compute embeddings and persist aligned IDs:

```
matrix = embed_texts([c.text for c in chunks], model=settings.embedding_model)
bundle = EmbeddingBundle(chunk_ids=[c.chunk_id for c in chunks], matrix=matrix)
persist_embedding_bundle(bundle, settings.processed_dir)
```

5. Build relation-aware graph + communities:

```
relation_edges = extract_relationship_edges(chunks)
graph = build_entity_graph(chunks, relation_edges=relation_edges)
partition = detect_communities(graph)
summaries = community_summaries(graph, partition)
```

Key input/output data contracts

- MedRecord (normalized doc):
 - `pmid: str`, `split: str`, `title: str`, `abstract: str`, `text: str`, `entities: list[EntityMention]`
- EntityMention:
 - `text: str`, `concept_id: str`, `semantic_type_ids: list[str]`, `offsets: list[list[int]]`
- ChunkRecord:
 - `chunk_id`, `pmid`, `split`, `chunk_index`, `text`, `title`, `entity_count`, `concept_ids`, `entity_texts`
- Embedding bundle persisted as:
 - `data/processed/chunk_embeddings.npy` (2D float matrix)
 - `data/processed/chunk_embedding_ids.json` (list of chunk IDs aligned with rows)
- Graph artifacts persisted in NB02:
 - `graphs/entity_graph.pkl`

- `graphs/community_partition.json` (mapping concept ID -> community ID)
 - `graphs/community_summaries.json` (list of community summary dicts)
 - `graphs/relation_edges.json` (serialized `RelationEdge` list)
-

3.3 Flow C: GraphRAG retrieval (Chroma and Pinecone)

Primary modules: `src/chroma_retriever.py`, `src/pinecone_retriever.py`, `src/graph_builder.py`

Step-by-step (Chroma path)

1. Index chunk vectors + metadata into Chroma collection.
2. At query time:
 - Dense channel: `vector_search(collection, query, top_k)`
 - Graph/entity channel: `concept_id_from_query + local_graph_expansion + entity_search`
 - Fusion: `reciprocal_rank_fusion({"vector": ..., "entity": ...}, top_k)`

Short retrieval payload row returned by retrievers:

```
{
  "id": "12345_c0003",
  "text": "chunk text...",
  "metadata": {"pmid": "12345", "concept_ids": "C001|C002", "chunk_index": 3},
  "score": 0.88,
  "source": "vector"
}
```

Step-by-step (Pinecone benchmark path)

1. `index_chunks_to_pinecone(chunks, embeddings, index_name, namespace)`
 2. Query with `query_pinecone(query, index_name, namespace, top_k)`.
 3. Use the same graph-aware rerank idea and RRF for parity against Chroma in `NB03_Pinecone_GraphRAG.py`.
 4. Persist benchmark report to `outputs/metrics/nb03_retrieval_benchmark.json`.
-

3.4 Flow D: Agentic GraphRAG request lifecycle

Primary module: `src/agentrag.py`

Definition This is a LangGraph stateful request flow with explicit routing, fallback, and grounding checks.

Step-by-step

1. Build resources (`AgentResources`) with Chroma collection, chunks, graph, partition, summaries.
2. Compile workflow via `build_agentic_workflow(resources)`.
3. Execute query via `run_agentic_query(app, query)`.
4. Node progression:
 - `retrieval`
 - `retrieval_grader`
 - conditional `web_search` or `graph_traversal`
 - `context_expansion`
 - `answer_generation`
 - `hallucination_detection` (may retry generation once)
 - `final_response`

AgentState keys include: - request: `query` - retrieval: `retrieved_docs`, `retrieval_score`, `extracted_concept_ids` - graph/web context: `graph_traversal`, `selected_communities`, `web_results`, `expanded_context` - generation safety: `answer_draft`, `hallucination_score`, `retries` - final: `final_answer`, `route`, `trace`

3.5 Flow E: CRAG corrective lifecycle

Primary module: `src/crag_pipeline.py`

Definition CRAG adds a strict quality gate and correction loop before accepting retrieved context.

Step-by-step

1. Build workflow with `CRAGResources(chroma_collection, sparse_index)`.
2. Execute with `run_crag_query(app, query)`.
3. Node progression:
 - `retrieve` (hybrid search)
 - `grade_retrieval`
 - conditional branch to `context_expansion`, `query_correction`, or `web_fallback`
 - `answer_generation`
 - `verify_answer`
 - conditional retry via `web fallback` or `finalize`

CRAGState key outputs include: - quality controls: `retrieval_grade`, `retrieval_reason`, `missing_aspects`, `groundedness`, `hallucination_risk` -

correction counters: `correction_attempts`, `verify_attempts` - route/trace:
route, trace - final text: `final_answer`

3.6 Flow F: Multimodal ingestion and retrieval

Primary modules: `src/multimodal_assets_pmc.py`, `src/multimodal_rag.py`,
`src/multimodal_vision_rag.py`

Step-by-step

1. Asset generation/fetch:
 - `fetch_pmc_multimodal_assets()` writes `data/multimodal/pmc_asset_manifest.json`.
2. OCR/table path (NB08/NB09):
 - `build_multimodal_documents(image_paths, table_paths, ...)`
 - `multimodal_documents_to_chunks(...)`
 - `index_multimodal_chunks_to_chromadb(...)`
3. Vision path (NB10):
 - `build_vision_multimodal_documents(...)`
 - `vision_documents_to_chunks(...)`
 - `index_vision_chunks_to_chromadb(...)`
4. Retrieval via `multimodal_vector_search` or `vision_multimodal_search`.

Key multimodal contracts: - `MultimodalDocument`: - `asset_id`, `modality` (image/table), `source_path`, `title`, `extracted_text`, `metadata` - `MultimodalChunk`:
- `chunk_id`, `asset_id`, `modality`, `chunk_index`, `text`, `metadata` - OCR backend
marker in metadata: - `ocr_backend` is `ollama_run` or `ollama_chat_fallback`
(or OCR failure path)

3.7 Flow G: Optional selective fine-tuning lifecycle (NB11)

Primary modules: `src/finetune_data.py`, `src/finetune_unsloth.py`

Step-by-step

1. Build SFT examples from real eval queries + supporting chunks:
 - `build_biomedical_sft_examples(...)`
2. Split and persist JSONL:
 - `train_eval_split_sft(...)`
 - `persist_sft_jsonl(...)` -> `sft_train.jsonl`, `sft_eval.jsonl`
3. If `RUN_FULL_TRAIN=true`:
 - create model path (`create_unsloth_lora_model` or `fallback_create_peft_lora_model_fallback`)
 - build trainer (`create_sft_trainer`)

- train + export adapters (`save_adapter_bundle`)
 - write Modelfile (`write_ollama_modelfile_template`)
4. If disabled or failed:
- write schema-complete placeholder reports (`persist_finetune_placeholder_report`)

Key persisted NB11 artifact shape (`outputs/metrics/nb11_selective_finetune_metrics.json`):

- `stack`, `dataset`, `lora_config`, `training_config`, `trainer_log_summary`, `adapter_metadata`, `comparison_payload`, `placeholder_report`

Module 4: Setup & Run Guide

4.1 Clean-machine setup (from repository scripts and README)

```
cd /home/ahmad/AI/Github/Medical-Research-GraphRAG
```

```
uv python install 3.12.10
uv venv --python 3.12.10 .venv
source .venv/bin/activate
uv sync --extra dev --extra finetune
```

4.2 Environment configuration

1. Create `.env` from `.env.example`.
2. Fill values relevant to your target notebooks.

Required vs optional (as implemented):

Variable	Where Used	Required For	Default/Behavior
PINECONE_API_KEY	<code>src/pinecone_retrieval.py</code> , strict/full scripts	NB03 and full pipeline	Required for Pinecone sections
PINECONE_CLOUD	<code>src/config.py</code>	Pinecone index creation	<code>aws</code>
PINECONE_REGION	<code>src/config.py</code>	Pinecone index creation	<code>us-east-1</code>
PINECONE_INDEX_PREFIX	<code>src/config.py</code> , NB03	Pinecone naming	<code>medmentions-graphrag</code>
OLLAMA_HOST	<code>embeddings/judge/agents.py</code>	<code>Antic/crag/evaluator/scripts/</code> , Ollama-backed stage	<code>http://127.0.0.1:11434</code> fallback in many paths
JUDGE_MODEL	<code>src/config.py</code>	judge functions	<code>granite4.1:8b</code>
GUARDIAN_JUDGE_MODEL	Models in settings	backward compatibility	<code>granite4.1:8b</code>
MULTIMODAL_OCR_MODEL	<code>src/config.py</code>	NB08/NB09 OCR	<code>glm-ocr</code>
MULTIMODAL_VISION_MODEL	<code>src/config.py</code>	NB10 vision	<code>qwen3.5:4b</code>
OCR_CLI_ALLOW_FALLBACK	<code>src/config.py</code>	OCR fallback behavior	<code>true</code>

Variable	Where Used	Required For	Default/Behavior
OCR_CLI_TIMEOUT_SECONDS	src/config.py	OCR CLI timeout	120
FINETUNE_* keys	src/config.py, NB11	optional fine-tune	defaults from <code>.env.example/settings</code>
RUN_FULL_EVAL	NB06-NB10, scripts	executes full metric paths	defaults false in execute scripts
RUN_FULL_TRAIN	NB11, scripts	executes training block	defaults false in execute scripts
CLEANUP_PINECONE_INDEX	scripts	optional index deletion	default false
MAX_RETRIES	strict script	retry policy	default 2
NOTEBOOK_TIMEOUT_SECONDS	strict script	nbconvert timeout	default -1
OLLAMA_EMBED_TIMEOUT_SECONDS	src/llm_embeddings.py	embedding timeout	default 900
OLLAMA_JUDGE_RETRIES	src/llm_judge.py	judge retry count	default 3
OLLAMA_JUDGE_TIMEOUT_SECONDS	src/llm_judge.py	judge timeout	default 90
OLLAMA_GENERATION_TIMEOUT_SECONDS	src/llm_generation.py	PyAG generation timeout	defaults 90/120
MULTIMODAL_OCR_CHAT_RETRIES	src/multimodal_rag.py	OCR fallback retries	default 2
MULTIMODAL_VISION_RETRIES	src/multimodal_vision_rag.py	vision rag retry retries	default 2

4.3 Dependency and service prerequisites

- Python 3.12.10 (enforced in `pyproject.toml`)
- uv package manager
- Ollama service and pulled models referenced by scripts:

```
ollama pull qwen3-embedding:4b
ollama pull granite4.1:8b
ollama pull glm-ocr
ollama pull qwen3.5:4b
```

4.4 Typical command sequences

Baseline notebooks only (NB01-NB05):

```
bash scripts/execute_notebooks.sh
```

Additive notebooks (NB06-NB11):

```
bash scripts/execute_additional_notebooks.sh
```

Strict end-to-end resumable pipeline:

```
bash scripts/run_full_real_pipeline_strict.sh
```

Fetch multimodal assets explicitly:

```
python scripts/fetch_pmc_multimodal_assets.py --max-images 5 --max-tables 3
```

4.5 Migration/seeding notes

- There is no relational DB migration framework in this repo.
- “Seeding” equivalents are file/artifact generation steps:
 - `medmentions_records.json` generation in NB01
 - chunk/embedding generation in NB02
 - graph artifact generation in NB02
 - multimodal manifest/assets generation via `scripts/fetch_pmc_multimodal_assets.py`

4.6 PDF export for this handbook

From repository root:

```
pandoc docs/zero_to_hero_study_handbook.md -o docs/zero_to_hero_study_handbook.pdf
```

Module 5: Study Plan & Practice Exercises

5.1 Ordered self-study plan

Recommended reading/exploration order for a new learner:

1. `README.md` and `pyproject.toml` (project goals, stack, dependency boundaries)
2. `src/config.py` (all runtime knobs and directory semantics)
3. `src/data_pipeline.py` + `src/chunking.py` (data contracts and pipeline foundations)
4. `src/embeddings.py` + `src/chroma_retriever.py` + `src/graph_builder.py` (core GraphRAG mechanics)
5. `notebooks/NB02_Chroma_GraphRAG.py` (puts all core foundations together)
6. `src/agent_rag.py` then `notebooks/NB04_Agentic_GraphRAG.py`
7. `src/evaluator.py` + `src/llm_judge.py` then `notebooks/NB05_Evaluation.py`
8. `src/hybrid_retriever.py` + `src/crag_pipeline.py` then NB06 and NB07
9. `src/multimodal_rag.py`, `src/multimodal_vision_rag.py` then NB08-NB10
10. Optional: `src/finetune_data.py`, `src/finetune_unsloth.py`, NB11

5.2 Practice exercises (with solution outlines)

Exercise 1 Question: Trace how one raw MedMentions record becomes one or more `ChunkRecords`. - Files: `src/data_pipeline.py`, `src/chunking.py`

Solution outline: - `load_medmentions_records` creates `MedRecord` with merged `title+abstract` and normalized entities. - `build_chunks` calls `recursive_split` and `_entities_for_chunk`. - Output chunk IDs are `"{pmid}_c{idx:04d}"` with metadata fields `concept_ids`, `entity_texts`.

Exercise 2 Question: Explain how GraphRAG combines semantic and entity channels in Chroma. - Files: `src/chroma_retriever.py`, `notebooks/NB02_Chroma_GraphRAG.py`

Solution outline: - Dense results from `vector_search`. - Entity-filtered results from `entity_search` using concept IDs. - Fused ranking via `reciprocal_rank_fusion` using `id` as merge key.

Exercise 3 Question: List every decision point in the agentic workflow and the variable controlling each decision. - Files: `src/agent_rag.py`, `src/config.py`

Solution outline: - Retrieval route decision: `retrieval_score < settings.retrieval_grade_threshold`. - Hallucination retry decision: `hallucination_score < settings.hallucination_threshold` with retry cap via `retries`.

Exercise 4 Question: In CRAG, what causes `query_correction` vs `web_fallback`? - Files: `src/crag_pipeline.py`, `src/config.py`

Solution outline: - If retrieval grade `>= crag_acceptance_threshold`: accept path. - Else if `correction_attempts < crag_max_corrections`: `query_correction`. - Else: `web_fallback`.

Exercise 5 Question: Reconstruct the schema of `outputs/metrics/nb05_evaluation_bundle.json` from code. - Files: `src/evaluator.py`, `notebooks/NB05_Evaluation.py`

Solution outline: - Top-level keys: `retrieval_metrics`, `generation_metrics`, `rag_metrics`, `metadata`. - Built by `EvaluationBundle.to_dict()` after `build_evaluation_bundle(...)`.

Exercise 6 Question: Compare OCR and vision multimodal ingestion contracts. - Files: `src/multimodal_rag.py`, `src/multimodal_vision_rag.py`

Solution outline: - Both produce `MultimodalDocument` and `MultimodalChunk`. - OCR path metadata includes `ocr_backend` and `ocr_model`. - Vision path metadata includes `vision_model` and `source_type=vision_model`.

Exercise 7 Question: Identify all places where runtime behavior changes based on `RUN_FULL_EVAL`. - Files: `notebooks/NB06_Hybrid_RAG.py`, `NB07_CRAG.py`, `NB08_Multimodal_RAG.py`, `NB09_Multimodal_RAG_OCR_CLI.py`, `NB10_Multimodal_RAG_Vision_Qwen.py`

Solution outline: - In each notebook, full retrieval/generation metric computation is gated. - When false, schema-complete placeholder payloads are persisted with `mode: placeholder`.

Exercise 8 Question: Explain why NB11 comparison payload can remain placeholder even when `RUN_FULL_TRAIN=true`. - Files: `notebooks/NB11_Selective_Finetuning_Unsloth_PEFT_TR`
`outputs/metrics/nb11_selective_finetune_metrics.json`

Solution outline: - Training path can fail (`trainer_log_summary.status == "failed"`). - If baseline/finetuned eval bundles are not both available, `comparison_payload.mode` remains placeholder. - Current artifact shows failure error 'LlamaAttention' object has no attribute 'apply_qkv'.

5.3 Final learner verification checklist

Use this to self-check understanding:

- Can you explain the exact end-to-end path in `scripts/run_full_real_pipeline_strict.sh` and how resume state works?
- Can you describe `MedRecord` -> `ChunkRecord` -> `embedding row` -> `graph node/edge` transformations with field names?
- Can you explain when Agentic GraphRAG routes to `web_search` and when it stays on `graph_traversal`?
- Can you explain CRAG correction-loop boundaries (`crag_max_corrections`, `verify_attempts`)?
- Can you reconstruct the schema of `nb05_evaluation_bundle.json` without opening it?
- Can you explain the difference between OCR multimodal metadata and vision multimodal metadata?
- Can you list which env vars are mandatory for Pinecone runs versus optional tuning vars?
- Can you explain why NB06-NB11 can emit placeholders and how to switch them to executed outputs?
- Can you point to where LoRA and SFT trainer configs are defined and exported in NB11?
- Can you describe one safe rollback path (for example: disable optional finetuning path and continue with baseline generator)?